

# NL2DSL - 自然语言到 DSL 智能问数系统

## 项目概述

NL2DSL 是一个企业级智能问数系统。核心思想：**AI 负责语义理解，系统负责执行治理。**

与传统 NL2SQL（自然语言直接转 SQL）不同，本系统采用分层架构：

```
自然语言 → LLM → DSL → 校验 → 权限注入 → 语义解析 → SQLAlchemy → 标准 SQL → 执行
```

这样做的好处：SQL 可校验、权限可控、查询可优化、多数据库方言可适配。

- **技术栈:** FastAPI + SQLAlchemy + sqlglot + Milvus Lite
- **部署形态:** 纯后端 API 服务（FastAPI）
- **LLM 接入:** 通义千问 API（默认），可配置切换其他兼容 OpenAI 接口的模型

## 快速开始

### 环境要求

- Python 3.10+

### 安装依赖

```
pip install -e ".[dev]"
```

### 配置环境变量

```
# 复制模板
# cp .env.example .env
```

```
# 编辑 .env 文件，填入 LLM API Key
NL2DSL_LLM_API_KEY=your-dashscope-api-key
```

## 启动应用

```
uvicorn nl2dsl.api:app --reload --host 0.0.0.0 --port 8000
```

## 运行测试

```
# 全部测试
pytest

# 只跑单元测试
pytest tests/unit/

# 带覆盖率
pytest --cov=nl2dsl --cov-report=html
```

## 验证服务

```
# Health 检查
curl http://localhost:8000/health

# 自然语言查询（LLM 优先，失败自动回退到关键词匹配）
curl -X POST http://localhost:8000/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{
    "question": "查询华东地区销售额最高的 10 个产品",
    "user_id": "u001",
    "tenant_id": "t001"
  }'

# 直接生成 DSL（不执行 SQL）
curl -X POST http://localhost:8000/api/v1/query/dsl \
  -H "Content-Type: application/json" \
  -d '{
    "question": "查询华东地区销售额",
```

```
"user_id": "u001",
"tenant_id": "t001"
}'
```

## 核心概念

### DSL（领域特定语言）

DSL 是 LLM 和系统之间的契约。LLM **只负责生成 DSL (JSON)**，不直接生成 SQL。

```
{
  "metrics": [{"func": "sum", "field": "order_amount", "alias": "sales_amount"}],
  "dimensions": ["product_name", "brand"],
  "filters": [
    {"field": "region", "operator": "=", "value": "华东"}
  ],
  "order_by": [{"field": "sales_amount", "direction": "desc"}],
  "limit": 10,
  "data_source": "orders",
  "joins": [
    {"table": "customer_dim", "on_field": "customer_id", "join_type": "left", "alias": "c"},
    {"table": "product_dim", "on_field": "product_id", "join_type": "inner", "alias": "p"}
  ]
}
```

关键约束： - `field` 必须是语义层已注册的维度名（不是数据库原始列名） - `metric` 必须是语义层已注册的指标名 - 无 `limit` 时系统自动注入默认 `limit: 100` - `joins` 可选，支持 `inner` / `left` / `right` 三种 join 类型 - 禁止 `SELECT *`

### 语义层

语义层通过 YAML 配置文件统一管理业务指标和数据模型：

```
# configs/metrics.yaml
metrics:
  sales_amount:
    expr: SUM(pay_amount)
```

```

description: "销售额（实付金额合计）"

dimensions:
  product_name:
    column: product_name
    description: "产品名称"

data_sources:
  orders:
    table: order_fact
    metrics: [sales_amount, gmV, order_count, avg_order_value, total_discount]
    dimensions: [product_name, brand, category, region, channel, customer_type]
    joins:
      product_dim:
        on: product_id
        type: inner
        alias: p
      customer_dim:
        on: customer_id
        type: left
        alias: c

```

**规则：**所有业务查询必须通过语义层，禁止直接引用数据库原始字段。

## 查询链路

### 用户请求

- API 层接收（提取 user\_id / tenant\_id）
- LLM 生成 DSL（带上下文 prompt，无 RAG 时直接用表结构 prompt）
  - LLM 调用失败 → 自动回退到关键词匹配的 Mock DSL 生成器
- DSL 校验（字段存在性、操作符合法性）
- 行级权限注入（自动添加用户可见范围过滤 + tenant\_id 隔离）
- 列级权限检查（敏感字段黑名单）
- 语义展开（指标名 → SQL 表达式, value\_map 转换）
- SQLAlchemy 构建标准 SQL（支持多表 JOIN）
- SQL 安全扫描
- 数据库执行
- 审计日志记录（含完整 trace 链路）
- 返回响应

# 目录结构

```

nl2dsl/
├── api.py                # FastAPI 应用入口，路由注册（主入口，含真实数据初始化）
├── api_factory.py        # FastAPI App 工厂（用于 E2E 测试注入 mock 数据）
├── config.py             # 配置管理（Pydantic Settings，支持 .env 文件）
├── dsl/
│   ├── models.py        # Pydantic DSL Schema 定义（含 Join 模型）
│   ├── validator.py      # DSL 校验器（字段存在性、操作符合法性）
│   └── builder.py        # DSL 构建辅助工具
├── llm/
│   ├── agent.py          # QueryAgent 工作流（LLM 调用 + 失败回退逻辑）
│   ├── prompts.py        # System Prompt / User Prompt 模板
│   └── client.py          # LLM API 客户端封装（OpenAI SDK，支持 DashScope 等）
├── rag/
│   ├── store.py          # 向量存储抽象层（Milvus Lite）
│   ├── embedder.py       # 文本嵌入（MockEmbedder，用于测试）
│   └── retriever.py       # 检索逻辑
├── permission/
│   ├── models.py         # 权限模型定义
│   ├── row_level.py      # 行级权限注入（自动添加用户可见范围过滤）
│   └── column_level.py   # 列级权限控制（敏感字段黑名单 + 脱敏规则）
├── planner/
│   ├── optimizer.py      # 查询优化规则（预留）
│   └── router.py          # 路由决策（预留）
├── semantic/
│   ├── registry.py       # 指标 / 维度 / 数据源注册中心（YAML 加载）
│   └── resolver.py        # 指标展开、value_map 转换
├── sql_engine/
│   ├── builder.py        # SQLAlchemy Core 表达式构建（支持多表 JOIN）
│   ├── scanner.py        # SQL 安全扫描
│   └── dialect.py         # sqlglot 方言转换
├── audit/
│   └── logger.py          # 审计日志记录（查询全过程持久化到 SQLite）
├── feedback/
│   └── collector.py       # 用户纠错反馈收集
├── utils/
│   └── logger.py          # 统一日志配置（控制台 + 文件 + 按天轮转）

tests/
├── unit/                 # 单元测试（DSL 校验、权限注入、SQL 构建）

```

```

├─ integration/          # 集成测试（完整链路测试）
├─ e2e/                  # 端到端测试（完整链路 + mock 数据）
│   ├── conftest.py      # E2E 测试 fixtures
│   ├── mock_data.py     # 模拟数据库数据生成器
│   ├── fixtures/        # 测试配置（metrics_test.yaml 等）
│   └─ test_end_to_end.py
├─ configs/
│   ├── metrics.yaml     # 指标定义（含多数据源和 JOIN 配置）
│   └─ permissions.yaml  # 权限配置模板
├─ logs/
│   ├── nl2dsl.log        # 全量日志（INFO 及以上）
│   └─ nl2dsl.error.log   # 仅错误日志（ERROR 及以上）

```

## 开发规范

### 代码风格

- 格式化: `ruff` （`ruff format .`）
- Lint: `ruff check .`
- 类型检查: `mypy --strict`
- 行长度: 100

### 导入规范

- 使用绝对导入 （`from nl2dsl.dsl.models import DSL`）
- 第三方库 → 标准库 → 本地模块，按字母排序

### 命名规范

- 模块/包: `snake_case`
- 类: `PascalCase`
- 函数/变量: `snake_case`
- 常量: `UPPER_SNAKE_CASE`

### 测试规范

- LLM 调用在测试中**必须 Mock**（避免产生 API 费用）
- 数据库测试使用 SQLite 内存库
- RAG 测试使用内存向量存储
- 测试函数命名： `test_功能_场景_预期结果`

错误处理

- 自定义异常继承自 `NL2DSLException`
- 异常必须包含 `error_code` （供前端/调用方识别）
- API 层统一捕获并转换为标准错误响应格式

```
# nl2dsl/exceptions.py
class NL2DSLException(Exception):
    error_code: str
    status_code: int = 500

class ValidationError(NL2DSLException):
    error_code = "VALIDATION_ERROR"
    status_code = 400

class PermissionError(NL2DSLException):
    error_code = "PERMISSION_DENIED"
    status_code = 403

class NotFoundError(NL2DSLException):
    error_code = "NOT_FOUND"
    status_code = 404
```

配置说明

所有配置通过环境变量 + `.env` 文件管理，前缀 `NL2DSL_`。

| 变量                              | 必填 | 说明                               |
|---------------------------------|----|----------------------------------|
| <code>NL2DSL_LLM_API_KEY</code> | 否  | LLM API 密钥。不配置时自动使用 Mock DSL 生成器 |

| 变量                       | 必填 | 说明                                                                               |
|--------------------------|----|----------------------------------------------------------------------------------|
| NL2DSL_LLM_BASE_URL      | 否  | 自定义 API 基础 URL，默认 <code>https://dashscope.aliyuncs.com/compatible-mode/v1</code> |
| NL2DSL_LLM_MODEL         | 否  | 模型名称，默认 <code>qwen-plus</code>                                                   |
| NL2DSL_VECTOR_STORE_TYPE | 否  | 向量存储类型： <code>milvus_lite</code> （默认）                                            |
| NL2DSL_MILVUS_URI        | 否  | Milvus Lite 本地文件路径，默认 <code>./milvus_lite.db</code>                              |
| NL2DSL_DB_URL            | 否  | 数据库连接串，默认 <code>sqlite:///./nl2dsl.db</code>                                     |
| NL2DSL_MAX_LIMIT         | 否  | 单次查询最大返回行数，默认 <code>10000</code>                                                 |
| NL2DSL_QUERY_TIMEOUT     | 否  | 查询超时（秒），默认 <code>30</code>                                                       |

```
# config.py 使用方式
from nl2dsl.config import settings

api_key = settings.llm_api_key
```

## API 接口

### 查询接口

| 方法   | 路径                                 | 说明                       |
|------|------------------------------------|--------------------------|
| GET  | <code>/health</code>               | 健康检查                     |
| POST | <code>/api/v1/query</code>         | 自然语言查询（LLM 优先，失败回退 mock） |
| POST | <code>/api/v1/query/dsl</code>     | 仅生成 DSL（不执行 SQL）         |
| POST | <code>/api/v1/query/execute</code> | 直接执行 DSL                 |

| 方法   | 路径                                     | 说明                |
|------|----------------------------------------|-------------------|
| GET  | /api/v1/schema                         | 获取语义层 Schema      |
| GET  | /api/v1/metrics                        | 获取指标列表            |
| POST | /api/v1/feedback                       | 提交纠错反馈            |
| GET  | /api/v1/admin/audit/queries            | 查询审计日志列表          |
| GET  | /api/v1/admin/audit/queries/{query_id} | 查询单条审计详情（含 trace） |

查询响应示例

```
{
  "status": "success",
  "data": [...],
  "dsl": {...},
  "sql": "SELECT ...",
  "execution_time_ms": 15
}
```

日志系统

日志模块位于 `nl2dsl/utils/logger.py` ，提供统一日志配置：

- **控制台输出:** INFO 及以上级别，实时查看
- **文件输出:** `logs/nl2dsl.log` ，按天自动轮转，保留 7 天
- **错误日志:** `logs/nl2dsl.error.log` ，仅 ERROR 及以上
- **格式:** 时间 | 级别 | 模块名 | 消息

在代码中使用：

```
from nl2dsl.utils.logger import get_logger

logger = get_logger("my_module")
```

```
logger.info("处理请求: %s", request_id)
logger.error("处理失败: %s", error_message)
```

## 调试指南

### 查看 LLM 生成的 DSL（不执行）

```
curl -X POST http://localhost:8000/api/v1/query/dsl \
  -H "Content-Type: application/json" \
  -d '{"question": "...", "user_id": "u001", "tenant_id": "t001"}'
```

### 查看审计日志（含完整 trace 链路）

```
# 列表
curl "http://localhost:8000/api/v1/admin/audit/queries?limit=5"

# 单条详情
curl "http://localhost:8000/api/v1/admin/audit/queries/{query_id}"
```

审计日志包含每个查询的完整执行 trace： - `dsl_generate` - DSL 生成（标记是否使用了 LLM） - `validate` - DSL 校验 - `row_permission_inject` - 行级权限注入 - `column_permission_check` - 列级权限检查 - `semantic_resolve` - 语义解析 - `sql_build` - SQL 构建 - `sql_scan` - SQL 安全扫描 - `sql_execute` - SQL 执行

### 查看日志文件

```
# 实时查看
tail -f logs/nl2dsl.log

# 只看错误
tail -f logs/nl2dsl.error.log
```

### 切换 LLM / Mock 模式

- 启用 LLM: 在 `.env` 中配置 `NL2DSL_LLM_API_KEY`

- **纯 Mock 模式**: 删除或不配置 `NL2DSL_LLM_API_KEY`，系统会自动使用关键词匹配生成 DSL

## 查看向量库内容

Milvus Lite 是本地文件存储，数据文件在 `./milvus_lite.db`。

## 常见开发任务

---

### 添加新指标

1. 编辑 `configs/metrics.yaml`
2. 添加指标定义 (expr、description)
3. 重启服务
4. 在 `tests/unit/test_semantic_registry.py` 添加测试

### 添加新的 JOIN 数据源

1. 在 `configs/metrics.yaml` 的 `data_sources.{source}.joins` 中添加 join 配置
2. 确保数据库中已创建对应的表和索引
3. 在 `nl2dsl/api.py` 的 `_mock_dsl_from_question` 中添加对应的关键词识别逻辑

### 调试 DSL 生成失败

1. 先调用 `/api/v1/query/dsl` 获取生成的原始 DSL
2. 检查 DSL 中的 `field` 是否在语义层注册
3. 检查 `operator` 是否在允许列表中
4. 查看日志文件 `logs/nl2dsl.log` 中的 LLM 原始输出

### 添加权限规则

1. 行级权限: 编辑 `configs/permissions.yaml`
2. 列级权限: 编辑 `configs/permissions.yaml` 的 `sensitive_columns`
3. 脱敏规则: 在 `configs/permissions.yaml` 的 `masking_rules` 中添加

## 技术决策记录

---

## 为什么 LLM 只生成 DSL 不生成 SQL?

- DSL 是结构化 JSON，可校验、可修正
- SQL 是自由文本，出错后难以定位和修复
- DSL 层级可做权限控制（在编译为 SQL 前注入过滤条件）
- DSL 可做查询优化（在编译前做重写）

## 为什么同时支持 LLM 和 Mock DSL 生成?

- **LLM 模式**: 理解能力强，支持复杂语义和模糊表达，但有 API 成本和延迟
- **Mock 模式**: 零成本、零延迟，适合开发和测试环境，以及 LLM 不可用的场景
- 系统默认先尝试 LLM，失败自动回退到 Mock，保证可用性

## 为什么用 SQLAlchemy Core 而不是 ORM?

- Core 提供表达式树级别的控制，适合动态 SQL 构建
- ORM 的隐式行为（延迟加载、关系导航）不适合查询编译场景
- Core 与 sqlglot 配合更自然（表达式树 → SQL 字符串 → 方言转换）

## 为什么用 SQLite 作为默认数据库?

- 零部署成本，适合开发和演示
- 文件级存储（`nl2dsl.db`），数据持久化
- 生产环境可通过 `NL2DSL_DB_URL` 切换为 MySQL/PostgreSQL 等